

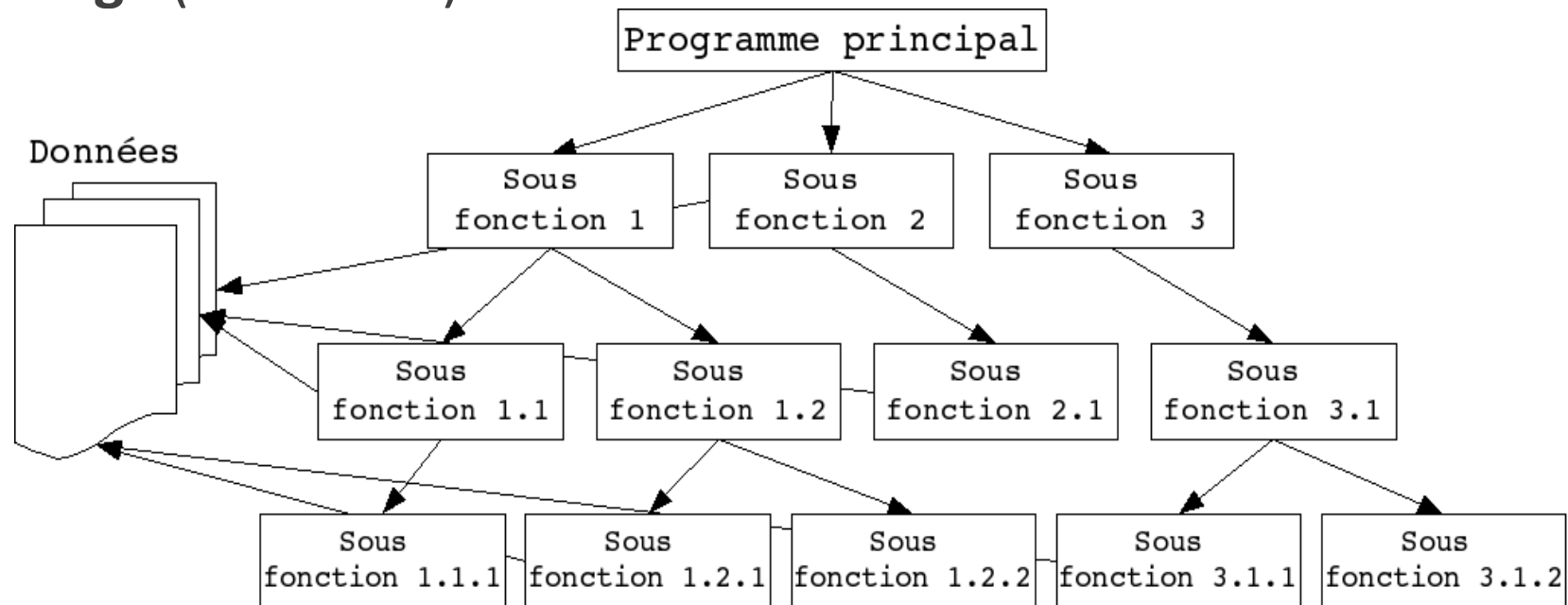
Analyse Orientée Objet

Méthode fonctionnelle

Système vu comme un **ensemble hiérarchique d'unités en interaction**.

Apect **traitement** est **priviliégié** (fonctions)

Les fonctions ont un état local mais l'**état du système est global et partagé** (centralisé).

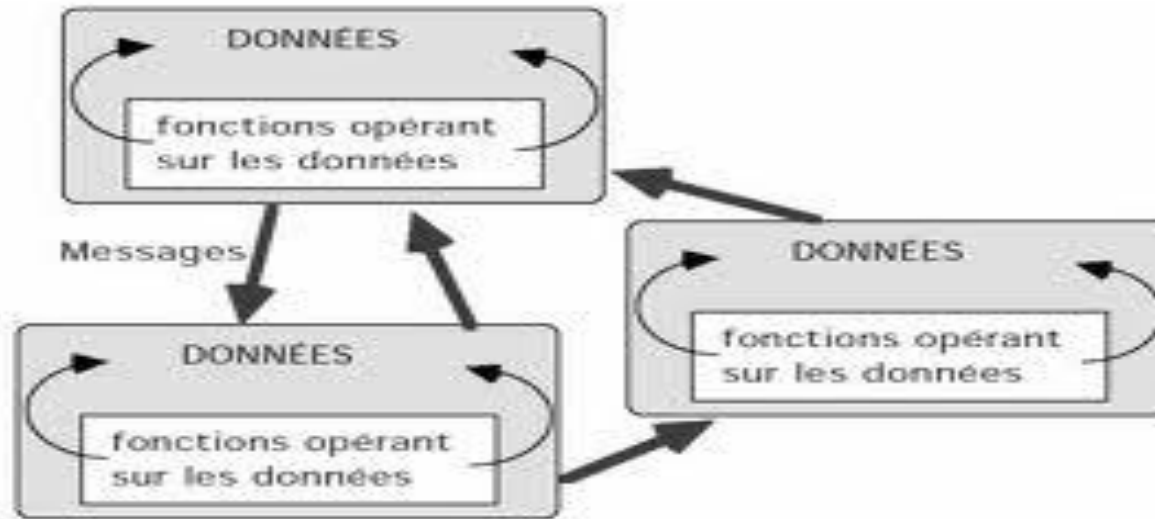


Méthode orientée objet

Système vu comme un **ensemble d'objet interagissant par des messages** (invocation de fonctions propres à chaque objet).

Chaque objet dispose d'un ensemble d'**attributs définissant son état**.

L'**état du système** est l'ensemble de l'état de tous ses objets (Décentralisé).



L'objet

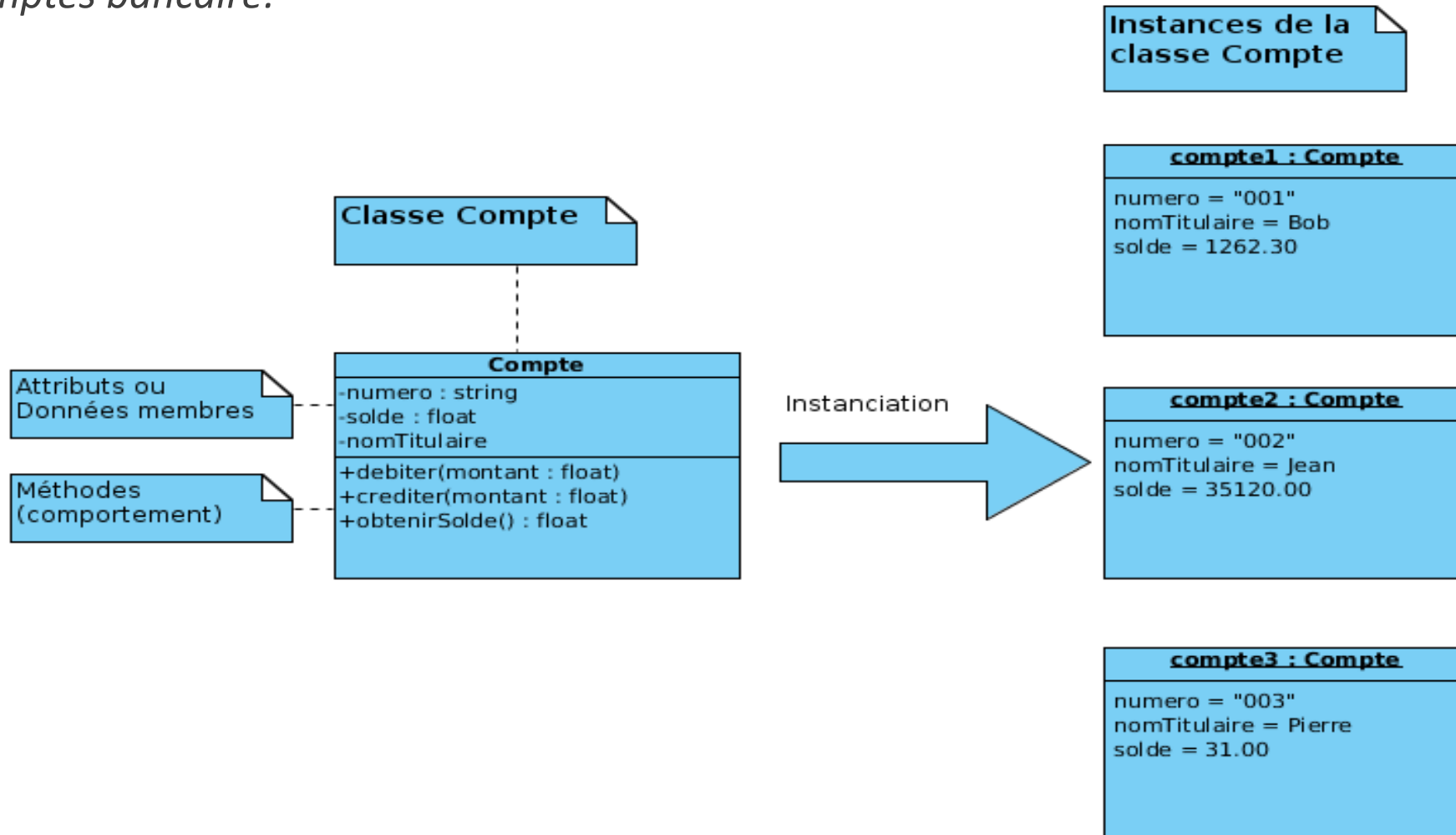
Un objet est une entité cohérente rassemblant des données et du code travaillant sur ses données.

Caractéristiques d'un objet:

- **Identité:** caractéristique qui permet de le différencier de tous les autres.
- **Etat:** situation à un instant constitué par l'ensemble de ses attributs et de ses liens.
- **Comportement:** ensemble des messages auquel un objet sait répondre.
- **Classe d'appartenance:** tout objet est une instance de sa classe (TDA moule à objet).

Exemple d'objets

Comptes bancaire:



Le pararadigme objet

”Un paradigme est une représentation du monde, ou une manière de voir les choses”

La conception avec des objets: accent mis sur les données et les actions qu'elles supportent.

Principes fondamentaux du paradigme objet:

- **L'encapsulation**
- **L'héritage**
- **Le polymorphisme**

Le principe d'encapsulation

Un objet rassemble en lui même ses données (**les attributs**) et les instructions capables d'agir dessus (**les méthodes**) .

Abstraction des données:

- Structure non visibles de l'extérieur.
- Seule son **interface** est connue et utilisable par un client

Abstraction procédurale:

- Invocation d'une méthode est atomique.
- Aucune information sur le mécanisme interne (Boite noire)

Apports du principe d'encapsulation :

Protection des données.

Liberté de modification de l'implantation des services (méthodes) ou des structures de données.

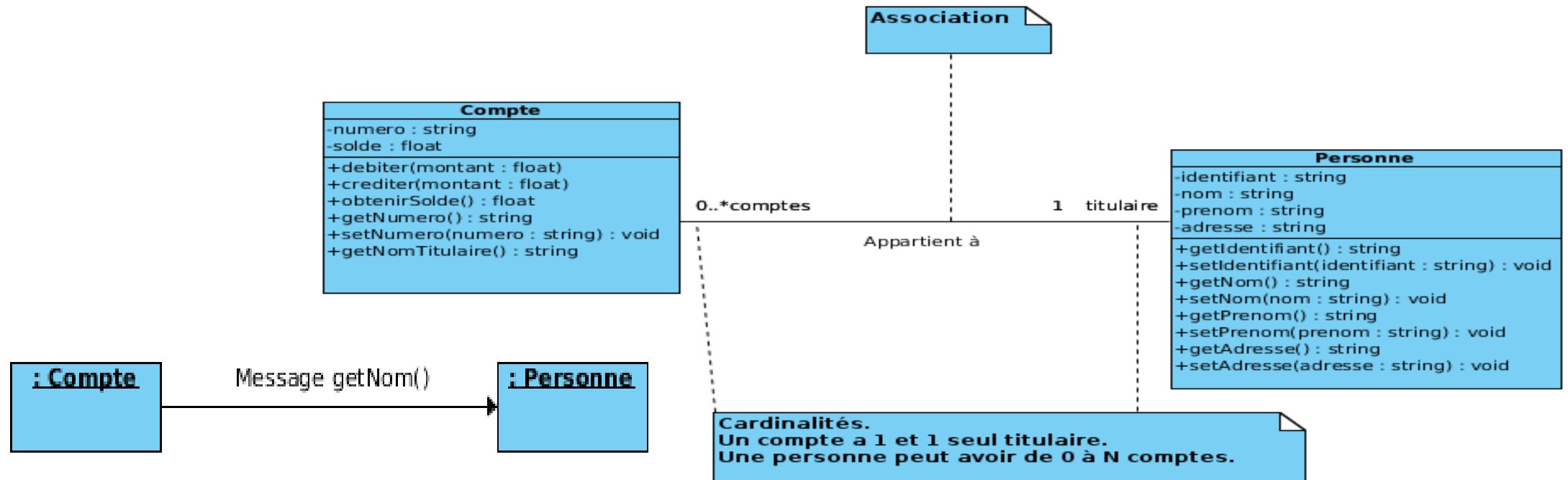
Le principe d'encapsulation

Exemple : Compte bancaire avec accesseurs et modifieurs pour lire et écrire les attributs encapsulés.

Compte
-numero : string -solde : float -nomTitulaire : string
+debiter(montant : float) +crediter(montant : float) +obtenirSolde() : float +getNumero() : string +setNumero(numero : string) : void +getNomTitulaire() : string

Le principe d'encapsulation

Création d'une classe « *Personne* » reliée au compte par l'**association** « *Appartient à* » en tant que titulaire du compte.



Le client peut continuer à invoquer la méthode *getNomTitulaire()* sans n'apercevoir aucun changement.

Classes et instances

Exercice :

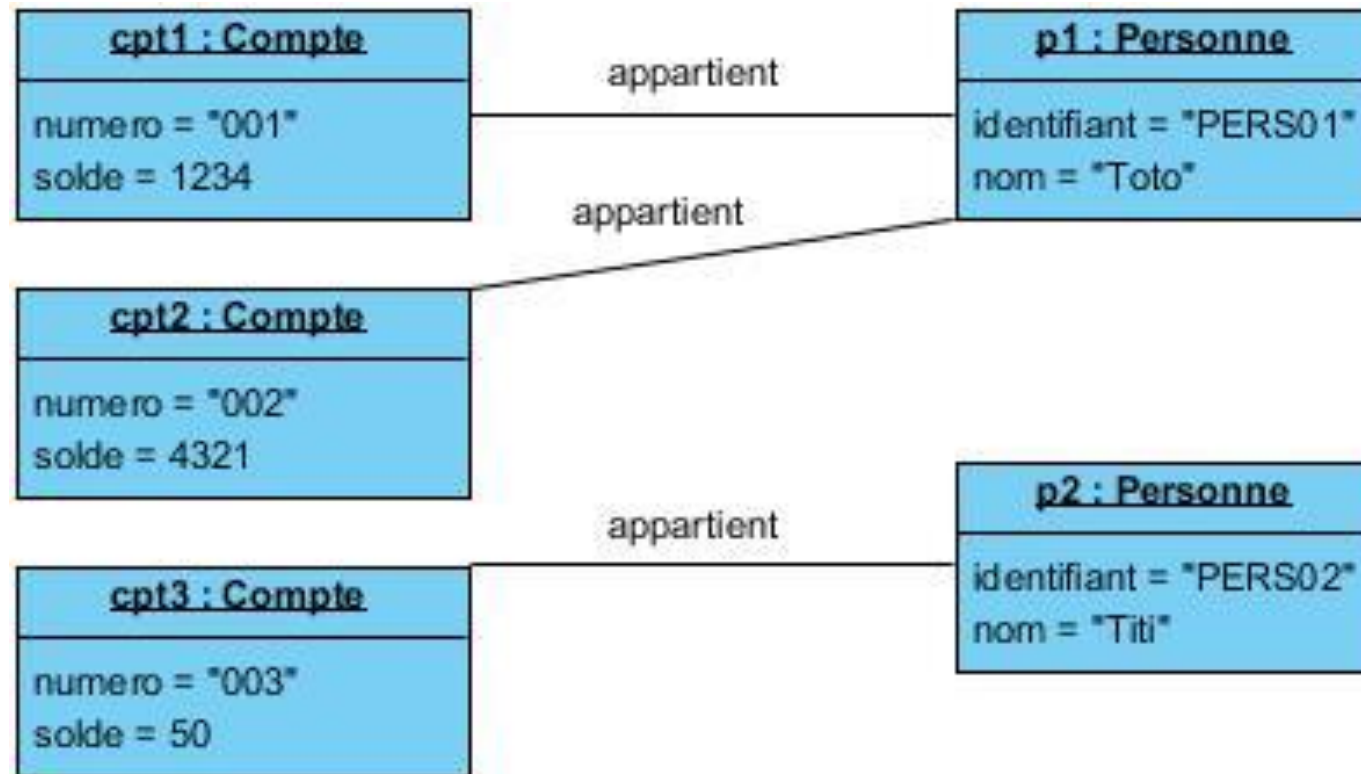
Dessiner un diagramme représentant des instances des classes « Compte » et « Personne ».

La relation qui doit exister entre une instance de Compte et son titulaire s'appelle un lien. Elle sera représentée graphiquement par une ligne allant du compte vers sa personne titulaire.

Classes et instances

Exercice :

Dessiner un diagramme représentant des instances des classes « Compte » et « Personne ».



Le principe d'héritage

Mécanisme de transmission des attributs et méthodes d'une **classe** (la **super-classe** ou classe mère) vers **une sous-classe** (classe fille ou classe dérivée).

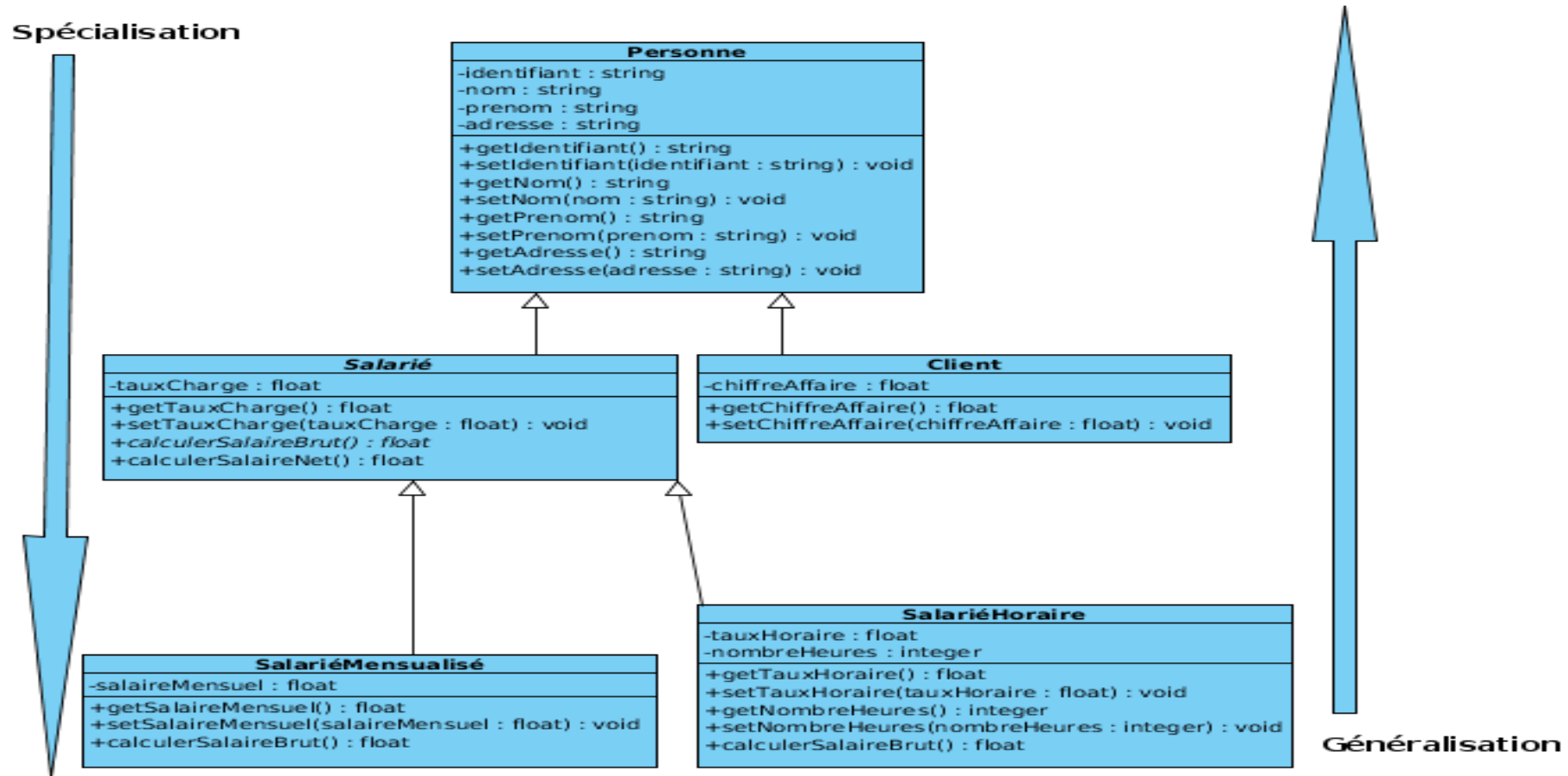
Principe naturel de **Généralisation / Spécialisation**.

Classification hiérarchique de concepts d'un niveau d'abstraction décroissant (arbre d'héritage).

Dans le cadre d'une spécialisation la sous-classe:

- Reprend le comportement de la super-classe.
- Peut redéfinir des comportements la super classe.
- Peut ajouter de nouveaux comportements.
- Ne peut supprimer aucun comportement de la super classe.

Le principe d'héritage



Un Client "est une" Personne. Un Salarié "est une" Personne.

Particularité de la classe Salarié ?

Particularité des méthodes calculerSalaireBrut() et calculerSalaireNet() de Salarié ?

Le principe d'héritage

Intérêt de l'héritage:

- Réutilisation
- Modélisation hiérarchique

Inconvénients de l'héritage:

- **Lecture plus complexe** : propriétés réparties sur plusieurs niveaux.
- **Manque de souplesse** : une sous-classe ne change pas de super-classe à l'exécution.
- Risque de conception trop détaillée si mal utilisé.

Le principe d'héritage

Classe Abstraite:

- Représente un **concept abstrait**.
- **Aucune instance** de la classe **ne peut exister**.
- Destinée à être la **super-classe** de **classes concrètes**
- Peut définir des méthodes sans implémentation : **méthodes abstraites** (à redéfinir par classes concrètes).

Une classification des méthodes:

- **Méthode concrète** : méthode implémentant la fonctionnalité pour laquelle elle a été prévue.
- **Méthode abstraite** : méthode sans implémentation qui doit être redéfinie dans les sous-classes concrètes.
- **Méthode générique** : méthode concrète d'une classe abstraite faisant souvent appel à des méthodes abstraites.

Le polymorphisme

Concept le plus puissant de l'orienté objet

- La méthode d'une classe sera invocable sur toutes ses sous-classes. (puisque héritée)
- La méthode peut **adopter une forme (comportement) différente** selon la sous-classe.

Obtenu par **redéfinition de la méthode dans les sous-classes** (adaptation du comportement).

Réalisé dans les langages objet par le principe de la **liaison dynamique ou liaison tardive**. (late binding)

- type statique / type dynamique
- Appel méthode **la plus proche du type dynamique de l'objet**.

Une forme faible du polymorphisme : la surcharge.

Exercice

Une entreprise possède un parc de véhicules regroupant : des voitures, des camions, des hélicoptères, des bateaux et des avions.

Elle souhaite modéliser le fonctionnement de son parc. Le but est de créer un système de classes permettant de factoriser le plus possible les fonctionnalités de chaque type de véhicule.

Partant des catégories à modéliser, il apparaît assez évident de dériver les classes Voiture et Camion d'une même classe VéhiculeRoulant et de laisser de côté les classes Hélicoptère et les Bateaux.

Bien que différents, tous les véhicules partagent certaines caractéristiques de par leur caractère mobile. Ainsi des actions telles que Démarrer et Arrêter possèdent un sens pour chaque type de véhicule. Aussi, toutes nos classes vont partager un ancêtre commun que nous appellerons Véhicule.

L'entreprise fait ensuite l'acquisition d'un Hydravion et d'une voiture amphibie. Il nous faut adapter la hiérarchie précédemment définie.

Proposer un diagramme de classe pour modéliser ce parc de véhicule.

Relations entre classes

Héritage : "est un"

Association: "utilise un" ou "a un"

- Relation structurelle entre objet.
- Chaque objet en association avec d'autres objets va jouer un rôle pour ces autres objets.
- Une association peut être bidirectionnelle ou unidirectionnelle.
- Notion de cardinalité N ou N..M

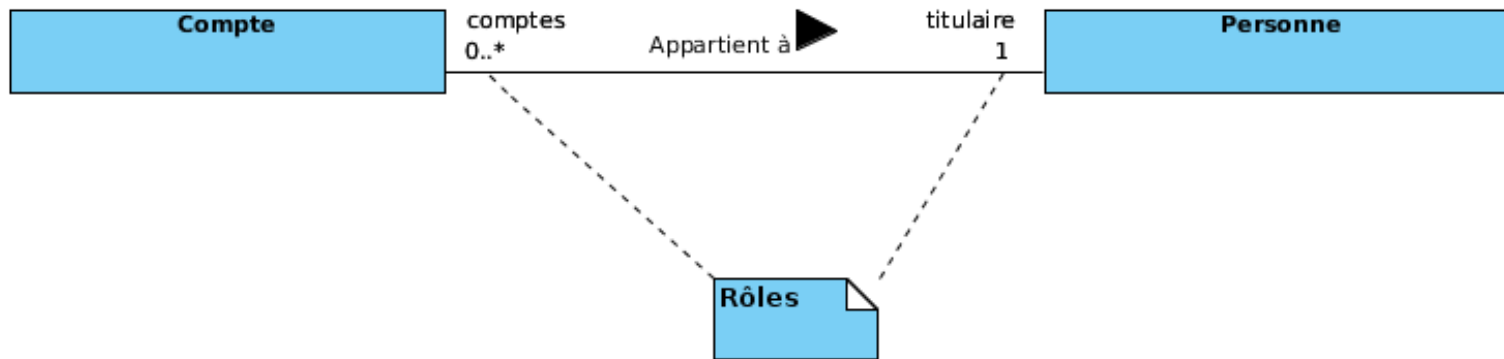
Agrégation : "est composé de"

- Forme d'association où un objet fait partie intrinsèque d'un autre objet (composite / composants)

Composition :

- Forme d'agrégation très forte, cycles de vies complètement liés.

Association

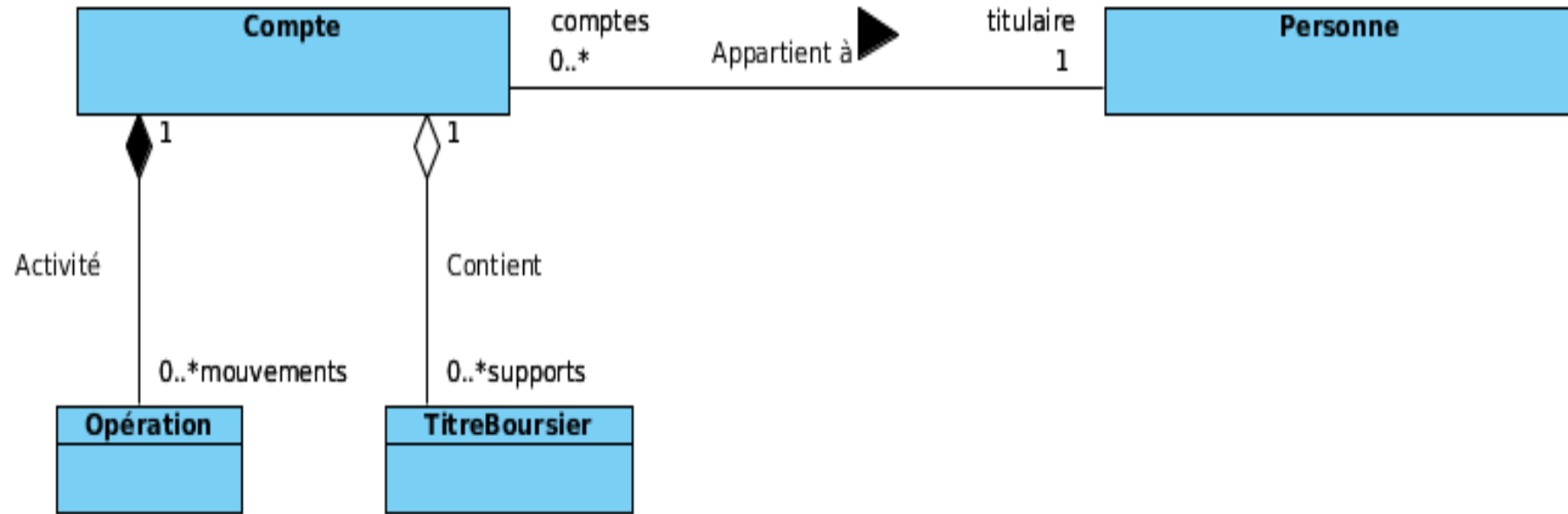


Association bidirectionnelle : chaque instance de compte connaît sa personne titulaire. Toute instance de personne connaît l'ensemble de ses comptes.



Association unidirectionnelle : comptes n'est pas navigable. Les personnes ne « connaissent » pas leurs comptes.

Agrégation et Composition



Un compte "est composé de" TitreBoursiers qui sont ses supports .

Un compte comporte ses Opérations. Si le compte est détruit, ses opérations n'ont pas lieu d'exister, elles seront détruites aussi.

Classe d'association

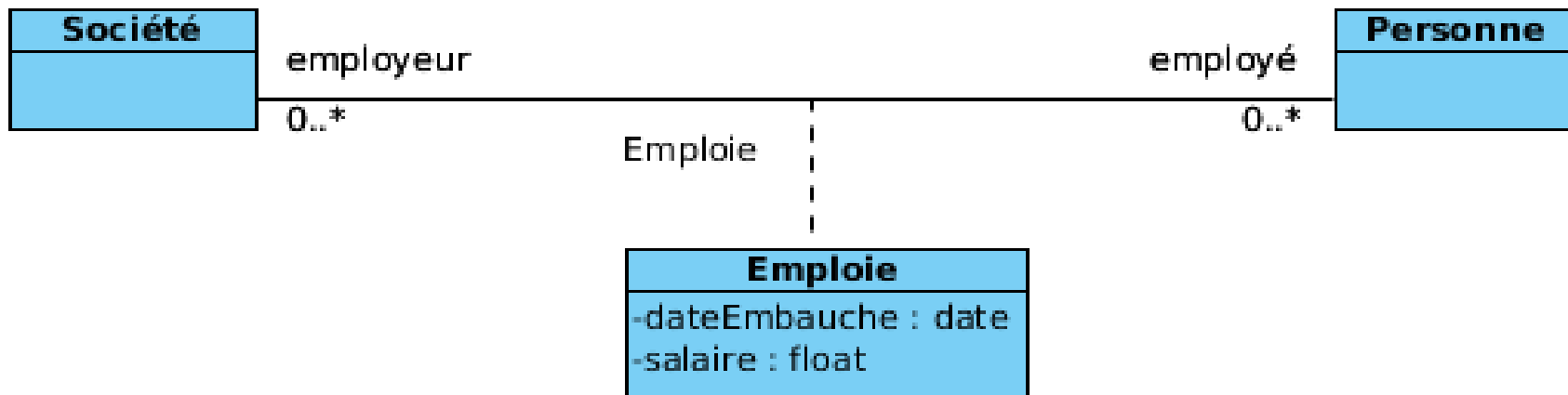


Une "Société" Emploie des "Personne".

Une "Personne" peut être employée par plusieurs "Société"

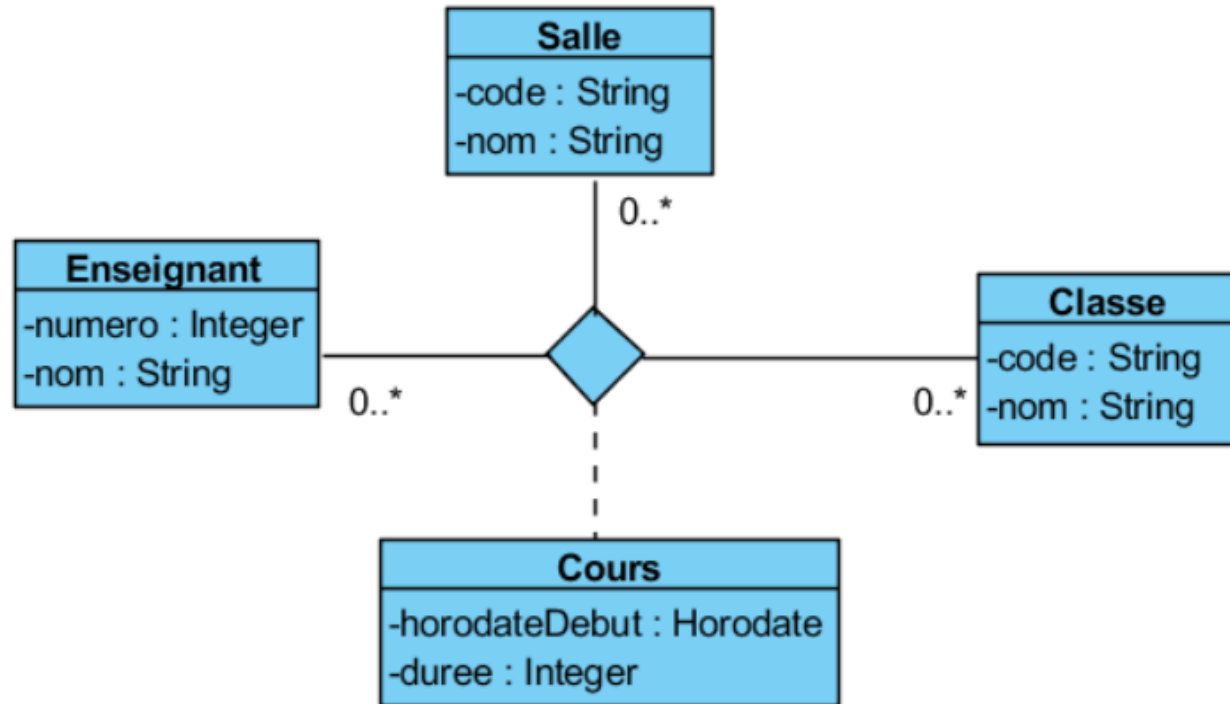
Où faire apparaitre la date d'embauche et le salaire d'un "employé" ?

*Lorsqu'une **association possède des attributs ou des opérations** elle peut être réifiée par une classe appelée **classe d'association**.*



Association n-aire

Une association entre plus de 2 classes est une association n-Aire



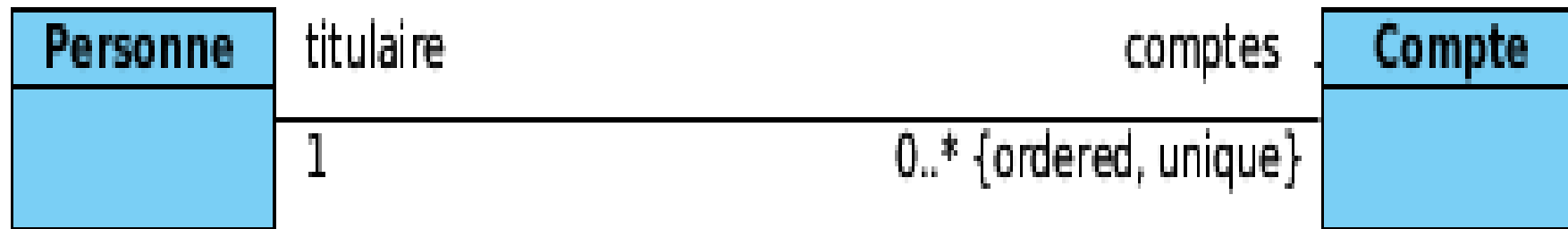
Une "Cours" a lieu à une date pendant une durée, il est donné à une "Classe" dans une "Salle" par un "Enseignant".

Contraintes sur les associations

Porte sur une relation ou sur un groupe de relations.

Notée **{*contrainte*}**

Exprime une règle à respecter dans la réalisation de l'association.



Il existe une relation d'ordre entre les comptes d'un titulaire {ordered}

L'association n'accepte pas plusieurs fois le même compte d'un titulaire {unique}.

Contraintes sur les associations

{unique} : le container est un **Set**

{non unique} : le container est un **Bag**

{ordered} : relation d'ordre entre les éléments

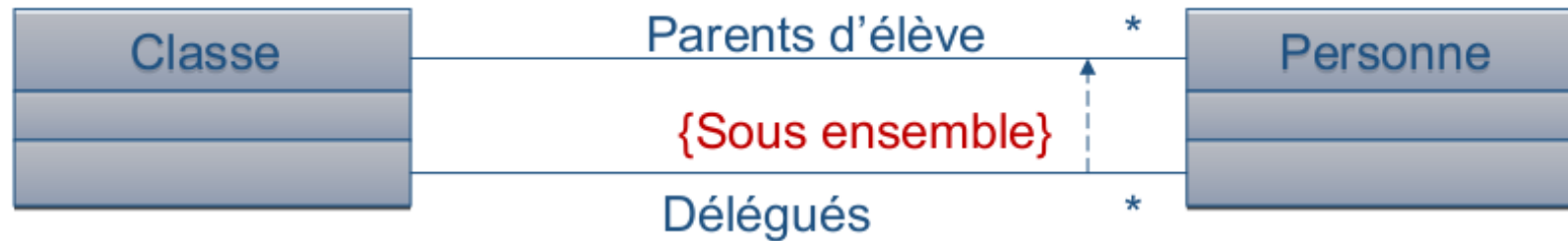
{unordered}: pas de relation d'ordre entre les éléments.

La combinaison de ces contraintes peut donner des containers:

Bag, Set, Ordered Bag, Ordered Set.

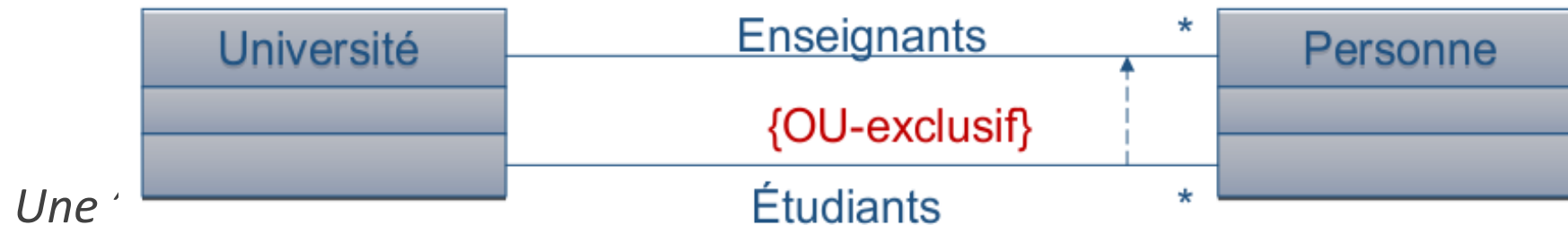
Contraintes sur les associations

{subset} ou **{sous ensemble}**: Une collection est incluse dans une autre.



Tout délégué de la "Classe" est une "Personne" parent d'élève de la "Classe".

{xor} ou **{ou exclusif}**: une seule association valide



Restriction des associations

Restriction ou **"Qualification"** d'une association.

Sélection d'un sous-ensemble d'objets parmi ceux qui participent à l'association.

Utilisation d'une **"Clé"** ou **"Qualifieur"**

Ensemble d'**attributs particuliers appartenant à l'association** et pas aux classes associées.

